Chapter Exercises

Federico

Exercise 1-1

> *Show that $a + b$ can be less than $\min(a, b)$.*

Let $a = -1$ and $b = -1$.

Then $a + b = -2$ and $\min(a, b) = -1$.

Since $-2 < -1$, the claim that $a + b$ can be less than $\min(a, b)$ holds.

Exercise 1-2

> *Show that $a \times b$ can be less than $\min(a, b)$.*

Let $a = -1$ and $b = 2$.

Then $a \times b = -2$ and $\min(a, b) = -1$.

Since $-2 < -1$, the claim that $a \times b$ can be less than $\min(a, b)$ holds.

Exercise 1-3

> *Design/draw a road network with two points a and b such that the fastest route between a and b is not the shortest route.*

We can realize such a network by using speed 'limits'– which should instead be treated as fixed speed specifications.

$$\begin{tikzcd}
a \arrow[d, dash, "\text{Route 2, at speed } s_2"'] \arrow[dr, "\text{Route 1, at speed } s_1"] & \\
c \arrow[r] & b
\end{tikzcd}$$

1. **Route 1 (direct):**
   - By the Pythagorean thourem, its length is $d_1 = \sqrt{2}l$.
   - Let its speed limit (and minimum speed) be $s_1 = v$.
   - The time to traverse it is $t_1 = \frac{d_1}{s_1} = \frac{\sqrt{2}}{v}l$

2. **Route 2 (through c):**
   - Its lenght is the sums of the lengths of paths a to c and c to b, whicth totals $d_2 = 2l$.
   - Let its speed limit (and minimum speed) be $s_2 = 2v$.
   - The time to traverse it is $t_2 = \frac{d_2}{s_2} = 2\frac{l}{2}v = \frac{l}{v}$

Conclusion:
While $d_1 < d_2$, it was shown that $t_1 > t_2$.

■

Exercise 1-4

> *Design/draw a road network with two points a and b such that the shortest route between a and b is not the route with the fewest turns.*

We can go from a to b by taking small deviations, which add turns but amount to little additional distance. This will yield a short path with a high number of turns. It is easy to create a second path with a small number of turns that deviates more from the straight-line path, yielding the desired network.

$$\begin{tikzcd}
a \arrow[r, "\text{Route 1}"] \arrow[ddr, "\text{Route 2}"'] & c_1 \arrow[d] & c_4 \arrow[r] & b \\
& c_2 \arrow[r] & c_3 \arrow[u] & \\
& & d \arrow[uur] &
\end{tikzcd}$$

1. **Route 1:**
   - It's easy to see that the total route length is $d_1 = 2l + l + 2l + l + 2l = 8l$.
   - Number of turns $b_1 = 4$.
2. **Route 2:**
   - By the Pythagorean thourem, its length is $d_2 = 2 \cdot \sqrt{2} \cdot 3l \approx 8.485l$.
   - Number of turns $b_2 = 1$.

Conclusion:
While $d_1 < d_2$, it was shown that $b_1 > b_2$.

■

Exercise 1-5

> *The* knapsack problem *is defined as follows: given a set of integers $S = \{s_1, s_2, ..., s_n\}$ and a target number T, find a subset of S that adds up exactly to T. For example, there exists a subset within $S = \{1, 2, 5, 9, 10\}$, that adds up to $T = 22$ but not $T = 23$.*
>
> *Provide counterexamples for the following algorithms for the knapsack problem. That is, give an S and T where the algorithm does not find a solution that leves the knapsack completely full, even though a full-knapsack solution exists.*
>
> *(a) Put the elements of S in the knapsack in left to right order if they fit, that is, the first-fit algorithm.*
> *(b) Put the elements of S in the knapsack from smallest to largest, that is, the best-fit algorithm.*
> *(c) Put the elements of S in the knapsack from largest to smallest.*

Solutions

These are all greedy heuristics, which can be tricked by constructing sets in ways that they make them choose elements which are not part of any solution for the problem.

(a) Left-to-right

Let $S = \{1, 2\}$ and $T = 2$. Picking the first element, 1, will leave no room for the second element, which would be the solution for this problem.

(b) Smallest to largest

Let $S = \{1, 2\}$ and $T = 2$. Picking the smallest element, 1, will leave no room for element 2, which would be the solution for this problem.

(c) Largest to smallest

A counter example for this case requires three elements. For example, let $S = \{2, 3, 4\}$ and $T = 5$. Picking the largest element, 4, will be outright wrong, as it is not part of the solution, $\{2, 3\}$.

■

Exercise 1-6

> *The* set cover problem *is as follows: given a set S of subsets $S_1, ..., S_m$ of the universal set $U = \{1, ..., n\}$, find the smallest subset of subsets $T \subseteq S$ such that $\bigcup_{t_i \in T} t_i = U$.*
>
> *For example, consider the subsets $S_1 = \{1, 3, 5\}$, $S_2 = \{2, 4\}$, $S_3 = \{1, 4\}$, and $S_4 = \{2, 5\}$. The set cover of $\{1, ..., 5\}$ would then be S_1 and S_2.*
>
> *Find a counterexample for the following algorithm: Select the largest subset for the cover, and then delete all its elements from the universal set. Repeat by adding the subset containing the largest number of uncovered elements until all are covered.*

First counterexample: tie on largest set

We can tie two subsets in number of elements, with only one of them being correct.

$$S = \{1, 2, 3\}, S_1 = \{1, 2\}, S_2 = \{2, 3\}, S_3 = \{3\}$$

There's nothing about that algorithm that will make us choose $\{2, 3\}$, which is part of the solution to this problem, $T = S_2 \cup S_3$.

Second counterexample: bad instance

We can trick the algorithm into picking a large subset that is not part of the solution.

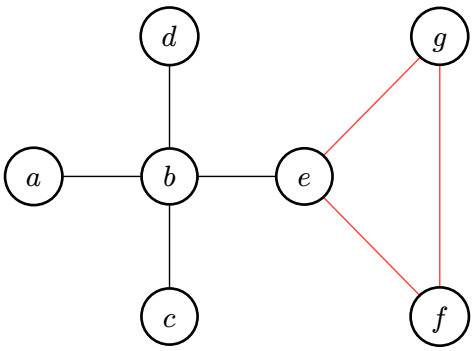
$$S = \{1, 2, 3, 4, 5, 6\}, S_1 = \{2, 3, 4, 5\}, S_2 = \{1, 2, 3\}, S_3 = \{4, 5, 6\}$$

The algorithm will always pick S_1, leaving it no choice but to pick up all subsets to cover S. The solution, instead is $T = S_2 \cup S_3$.

Exercise 1-7

> *The* maximum clique *problem in a graph $G = (V, E)$ asks for the largest subset C of vertices V such that there is an edge in E between every pair of vertices in C.*
>
> *Find a counterexample for the following algorithm:*
> 1. *Sort the vertices of G from highest to lowest degree.*
> 2. *Considering the vertices in order of degree, for each vertex, add it to the clique if it is a neighbor of all vertices currently in the clique.*
> 3. *Repeat until all vertices have been considered.*

If we consider the fact that this algorithm will always pick the highest-degree vertex, we can construct a graph in which such a vertex is not part of the maximum clique.



The algorithm will pick b first, which is not part of the solution, making it impossible to arrive to $C = \{e, g, f\}$.

Exercise 1-8

> *Prove the correctness of the following recursive algorithm to multiply two natural numbers, for all integer constants $c \geq 2$.*

$$
\begin{aligned}
&\text{Multiply}(y, z)\\
&\quad \text{if } \ z = 0 \ \text{ then return}(0) \ \text{ else}\\
&\quad \text{return}\left(\text{Multiply}\left(cy, \left\lfloor \frac{z}{c} \right\rfloor\right) + y \cdot (z \bmod c)\right)
\end{aligned}
$$

Proof

If the algorithm were correct, we must prove that $\text{Multiply}(y, z) = yz$.

Base case:

Let $z = 0$. By definition, $\text{Multiply}(y, 0) = 0$. Since $y \cdot 0 = 0$, the identity holds.

Induction:

Assume that for some $n > 0$, $\text{Multiply}(y, k) = yk$ holds for all $0 \leq k < n$. By definition:

$$\text{Multiply}(y, n) = \text{Multiply}\left(cy, \left\lfloor \frac{n}{c} \right\rfloor\right) + y(n \bmod c)$$

Applying the inductive hypothesis:

$$\text{Multiply}(y, n) = (cy)\left\lfloor \frac{n}{c} \right\rfloor + y(n \bmod c)$$

By the equivalence $\left\lfloor \frac{z}{c} \right\rfloor = \frac{z - z \bmod c}{c}$,

$$
\begin{aligned}
\text{Multiply}(y, n) &= (cy)\frac{n - n \bmod c}{c} + y(n \bmod c)\\
&= y(n - n \bmod c) + y(n \bmod c)\\
&= y(n - n \bmod c + n \bmod c)\\
&= yn
\end{aligned}
$$

Termination:

Given that $c \geq 2$, the sequence $z_{i+1} = \left\lfloor \frac{z_i}{c} \right\rfloor$ is strictly decreasing for all $z > 0$.

■

Exercise 1-9

Prove the correctness of the following algorithm for evaluating a polynomial $a_n x^n + a_{n-1}x^{n-1} + ... + a_1 x + a_0$.

$$
\begin{aligned}
&\text{Horner}(a, x)\\
&\quad p = a_n\\
&\quad \text{for } \ i \ \text{ from } \ n - 1 \ \text{ to } \ 0\\
&\quad\quad p = px + a_i\\
&\quad \text{return } \ p
\end{aligned}
$$

My original attempt

By working through a few iterations, it is clear that the algorithm does the right thing. However, I'm having a hard time with the proof. This much I know:

$$\text{Horner}(a, x) = p(0)$$

Here, $p(i)$ is the value of variable p at iteration i. Perhaps confusingly, the value of i dercreases after each iteration. Once it reaches 0, we have our solution.

Transcribing from the pseudocode,

$$p(i) = p(i+1)x + a_i$$

Therefore, $p(0) = p(1)x + a_0$.

We would like to assume that $p(1) = \frac{a_1 x + a_2 x^2 + \ldots + a_n x^n}{x}$. In other words:

$$p(i) = \frac{\sum_{k=i}^{n} a_k x^k}{x^i}$$

If we take this closed-form solution to be correct, then we can prove that:

$$p(i) = \sum_{k=i}^{n} a_k x^{k-i}$$

Extracting the first term from the sum and rearranging the second to have the shape we want:

$$\begin{aligned}
p(i) &= a_i + \left(\sum_{k=i+1}^{n} a_k x^{k-i} \right) \frac{x}{x} \\
&= a_i + \left(\sum_{k=i+1}^{n} a_k x^{k-i-1} \right) x \\
&= a_i + \left(\sum_{k=i+1}^{n} a_k x^{k-(i+1)} \right) x \\
&= a_i + p(i+1)x
\end{aligned}$$

Which has the exact property we were looking for. I guess this, combined with the fact that the series terminates at $p(n) = a_n$ by definition, is enough. ■?

After taking a peek at a few ways of solving this problem

It turns out I wasn't that far off. I only needed a structured approach that seems to be useful for this kinds of situations: Loop invariants. It seems that loop invariants are proven in three steps:

- Proving the invariant holds at the beginning (Initialization)
- Proving it holds between iterations (Maintenance)
- Proving it holds after the last iteration (Termination)

Now, my version:

Loop invariant

By looking at the algorithm, it can be seen that

$$\text{Horner}(a, x) = p(0)x + a_0$$

Where $p(0)$ is the value of the program variable p at the start of the last iteration. Generalizing, we say that $p(i)$ is the value of the program variable p at the start of iteration i.

$$p(i) = p(i+1)x + a_{i+1}$$

Note: iterations are enumerated by the value that the program variable i takes on each iteration. Therefore, as the loop progresses, i decreases. If we want to talk about the previous iteration, we must refer to $i+1$.

Note 2: We are focusing on the value of p at the *start* of each iteration and not at the end, because it makes slightly easier to check the invariant holds in the initialization case.

By manually working out a few more terms of $p(i)$, (we arrive at something like $\text{Horner}(a, x) =$), we can make an educated guess at the shape of the loop invariant:

$$\begin{aligned} p(i) &= \big(p(i+2)x + a_{i+2}\big)x + a_{i+1} \\ &= p(i+2)x^{i+2} + a_{i+2}x + a_{i+1} \\ &= \sum_{k=i+1}^{n} a_k x^{k-(i+1)} \end{aligned}$$

Now we need to prove that this invariant holds throught the entire loop.

Initialization

At the start of the loop, we know that $p = a_n$.

$$\begin{aligned} p(n-1) &= \sum_{k=(n-1)+1}^{n} a_k x^{k-((n-1)+1)} \\ &= \sum_{k=n}^{n} a_k x^{k-n} \qquad\qquad = a_n \end{aligned}$$

The invariant holds in this case.

Maintenance

We must show that the invariant holds from iteration to iteration. That is, calculating the next iteration must be consistent with the invariant equation.

$$\begin{aligned} p_{i+1} &= \sum_{k=(i+1)+1}^{n} a_k x^{k-((i+1)+1)} \\ &= \sum_{k=i+2}^{n} a_k x^{k-(i+2)} \end{aligned}$$

$$\begin{aligned} p_i &= p_{i+1}x + a_{i+1} \\ &= \left(\sum_{k=i+2}^{n} a_k x^{k-(i+2)}\right)x + a_{i+1} \\ &= \sum_{k=i+2}^{n} a_k x^{k-(i+1)} + a_{i+1} \end{aligned}$$

But the right term is exactly what we would get if we evaluated the left-term summation at $k = i+1$.

$$a_{i+1}x^{(i+1)-(i+1)} = a_{i+1}$$

Therefore, we can include it in the summation and simplify:

$$p_i = \sum_{k=i+1}^{n} a_k x^{k-(i+1)}$$

Which which shows that the invariant holds between iterations.

Termination

We must show that p after the loop yields the polynomial summation:

$$\begin{aligned} p_{\text{end}} &= p_0 x + a_0 \\ &= \left(\sum_{k=1}^{n} a_k x^{k-1} \right) x + a_0 \\ &= \sum_{k=1}^{n} a_k x^k + a_0 \end{aligned}$$

Similarly to how it was shown in the maintenance case, the last term can be expressed as the first term of the summation:

$$p_{\text{end}} = \sum_{k=0}^{n} a_k x^k$$

Which is the polynomial equation.

■

Exercise 1-10

> *Prove the correctness of the following sorting algorithm.*
>
> ```
> Bubblesort(A)
>  for i from n to 1
>  for j from 1 to i − 1
>  if (A[j] > A[j + 1])
>  swap the values of A[j] and A[j + 1]
> ```

The inner loop does the following operation for iteration j. $A_j, A_{j+1} = \min(A_j, A_{j+1}), \max(A_j, A_{j+1})$

Therefore $A_j \geq A_{j+1}$.

Invariant

I claim that this maintains the invariant $A_j = \max(A_{1:j})$. In consequence, the outer loop invariant is $A_i = \max(A_{1:i})$. This outer loop invariant has the desirable property that, after all iterations, $A'_1 \leq A'_2 \leq ... \leq A'_{n-1} \leq A'_n$.

Note: I'm using the notation $\max(A_{1:i})$ to mean "the maximum value of all elements from A_1 up to and including A_i".

Initialization
When $j = 1$, according to the invariant, $A_1 = \max(A_{1:1}) = A_1$, so the invariant holds in this case (a single item array is sorted by definition).

Maintenance
When $j = k$, according to the invariant, $A_k = \max(A_{1:k})$. To calculate A_{k+1}, we use the formula:

$$\begin{aligned} A_{k+1} &= \max(A_k, A_{k+1}) \\ &= \max(\max(A_{1:k}), A_{k+1}) \\ &= \max\left(A_{1:(k+1)}\right) \end{aligned}$$

The invariant holds in this case.

Termination
When $j = i - 1$:

$$\begin{aligned} A_i &= \max(A_{i-1}, A_i) \\ &= \max\left(\max\left(A_{1:(i-1)}\right), A_i\right) \\ &= \max(A_{1:i}) \end{aligned}$$

The outer loop invariant holds in this case.

■

Exercise 1-12

> *Prove that $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ for $n \geq 0$, by induction.*

Let us define $f(n)$ as $f(n) =: \frac{n(n+1)}{2}$.

Base case
Let $n = 0$. The summation yields $\sum_{i=1}^{0} i = 0$
$f(0) = \frac{0(0+1)}{2} = 0$.
The basis case is correct.

Inductive step
Assume the proposition holds for an arbitrary integer $k \geq 0$, such that $\sum_{i=1}^{k} i = f(k)$. We must show it holds for $k + 1$:

$$
\begin{aligned}
\sum_{i=1}^{k+1} i &= \sum_{i=1}^{k} i + (k+1) \\
&= f(k) + (k+1) \\
&= \frac{k(k+1)}{2} + (k+1) \\
&= \frac{k(k+1) + 2(k+1)}{2} \\
&= \frac{(k+1)(k+2)}{2} \\
&= \frac{(k+1)((k+1)+1)}{2} = f(k+1)
\end{aligned}
$$

The general case holds.

■

Exercise 1-13

> *Prove that* $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$ *for* $n \geq 0$*, by induction.*

Let $f(n) := \sum_{i=1}^{n} i^2$ and $g(n) := \frac{n(n+1)(2n+1)}{6}$.

Proposition: $f(n) = g(n)$ for $n \geq 0$.

Proof

Base case

Assume $n = 0$. Then $f(0) = g(0) = 0$.

Induction case

Assume the proposition holds for $n = k$, where $k \geq 0$. Then we must prove it also holds for $n = k + 1$.

$$
f(k+1) = \sum_{i=1}^{k+1} i^2 = \sum_{i=1}^{k} i^2 + (k+1)^2
$$

By our inductive hypothesis, we can substitute $\sum_{i=1}^{k} i^2$ by $g(k)$. Therefore, $f(k+1) = g(k) + (k+1)^2$. To complete the proof, we must show that $f(k+1) = g(k+1)$.

$$
g(k+1) = \frac{(k+1)((k+1)+1)(2(k+1)+1)}{6} = \frac{(k+1)(k+2)(2k+3)}{6}
$$

$$
\begin{aligned}
g(k) + (k+1)^2 &= \frac{k(k+1)(2k+1)}{6} + (k+1)^2 \\
&= \frac{k(k+1)(2k+1) + 6(k+1)^2}{6} \\
&= \left(\frac{k+1}{6}\right)(k(2k+1) + 6(k+1)) \\
&= \left(\frac{k+1}{6}\right)(2k^2 + 7k + 6)) \\
&= \left(\frac{k+1}{6}\right)(k+2)(2k+3) \\
&= \frac{(k+1)(k+2)(2k+3)}{6} = g(k+1)
\end{aligned}
$$

■

Exercise 1-19

> *Prove by induction that a tree with n vertices has exactly $n-1$ edges.*

Proposition
Let $s(n)$ represent the number of edges in a tree with n vertices, where n is a positive, natural number. We must prove $s(n) = n - 1$ for all $n > 0$.

Proof
For the **base case** is trivial to observe that a tree of one vertex has zero edges, so the proposition $s(1) = 1 - 1 = 0$ holds.

For the **inductive case**, assume the proposition holds for all trees of $n = k$, with $k > 0$. We must prove it also holds for a tree of $k + 1$ vertices.

Take an arbitrary tree of $k + 1$ vertices ($k + 1 \geq 2$). Any tree with more than one vertex has at least one leaf. Remove a leaf of this tree along its single incident edge. Because we only removed a leaf and its edge, we are left with an acyclic, connected graph – a tree – of k vertices.

Applying our hypothesis, we know that this tree must have $s(k) = k - 1$ edges. Restoring the removed leaf and edge, we must have a tree with one more edge, $k - 1 + 1 = k$. But this is the same as $s(k + 1)$. This proves our proposition holds for an arbitrary tree of $k + 1$ vertices.

■